Graph Based Intelligent Movie Recommendation System Using RAG

Project Report

Presented To
Prof. Kaikai Liu

Department of Software Engineering
San José State University

Submitted By
Venkata Siva Sai Krishna Prasad Yedupati-
018320835-venkatasivasaikrishnprasad.yedupati@sjsu.edu
Bharath Kumar A-018221268- bharathkumar.a@sjsu.edu
Sriya Verma Saripella - 019130553- sriyavarma.saripella@sjsu.edu

Project Track: System / Application

Focused Areas:
Knowledge Graphs, Retrieval-Augmented Generation, Recommender Systems, Semantic Search, FAISS, Large Language Models

In Partial Fulfillment of the Requirements for the Course

CMPE 258: Deep Learning

Spring 2026

May 2026

Table of Contents

I. INTRODUCTION

Traditional movie recommendation systems, such as collaborative filtering and content-based filtering, often suffer from significant limitations including popularity bias, cold-start issues, and a lack of interpretability. Collaborative filtering depends heavily on user-item interaction data, which is sparse for new users or movies. Content-based filtering makes its recommendations based on metadata like genres or actors, but fails to model complex relationships between entities.

To overcome these challenges, we proposed a hybrid approach that combines Knowledge Graphs (KGs) and Retrieval-Augmented Generation (RAG). We used Neo4j to retrieve the relationships between movies, genres, actors, users, and directors as a semantic knowledge graph. This structured information is converted into human-readable triplets and embedded for semantic retrieval. These embeddings are stored in a FAISS index, and a language model (LLaMA3 via Groq) is used to answer natural language queries based on the retrieved knowledge.

Our contributions include:

• Integrating knowledge graph triplets with Retrieval-Augmented Generation (RAG) to enable structured and explainable movie recommendations

• Designing a FAISS-based semantic retrieval pipeline over 200K+ knowledge graph triplets for efficient similarity search

• Implementing a hybrid retrieval mechanism combining vector search and cosine re-ranking for improved relevance

• Evaluating system performance using standard information retrieval metrics (Precision@K, Recall@K, F1-score) across multiple query types

II. RELATED WORK

Knowledge Graphs have become increasingly popular when it comes to recommendation systems due to their ability to represent structured relationships between entities. Nair and Cheriyan [4] applied particle filtering on multi-featured movie KGs to refine recommendation quality. This research demonstrates that traversing graph structures could uncover more meaningful user preferences.

Retrieval-Augmented Generation (RAG) was introduced by Lewis et al. [1] and has enabled more context-aware natural language responses by retrieving supporting documents during the

generation process. Xu et al. [5] extended this idea by integrating KGs into the retrieval process and improved customer support applications through fact-grounded answers.

The work we did involves both topics by constructing a Neo4j-based knowledge graph, extracting interpretable (subject-predicate-object) triplets, and indexing them using FAISS. We then used Groq's LLaMA3-70B language model to answer user queries with natural language explanations that come from retrieved facts. This pipeline allows our system to reason over both structural graph data and unstructured queries. This helped enhance the recommendation precision and transparency.

III. DATASET

The dataset we chose was from Neo4j's public "recommendations" knowledge graph. It contains 28,865 nodes and 166,262 relationships.

Key Node Types:

- Movie: Includes metadata such as title, year, IMDb rating, budget, revenue, runtime, and language.
- Actor and Director: Include names, roles, and birth/death years.
- User: Includes userId and rating history.
- Genre: Used to classify movies.

Relationship Types:

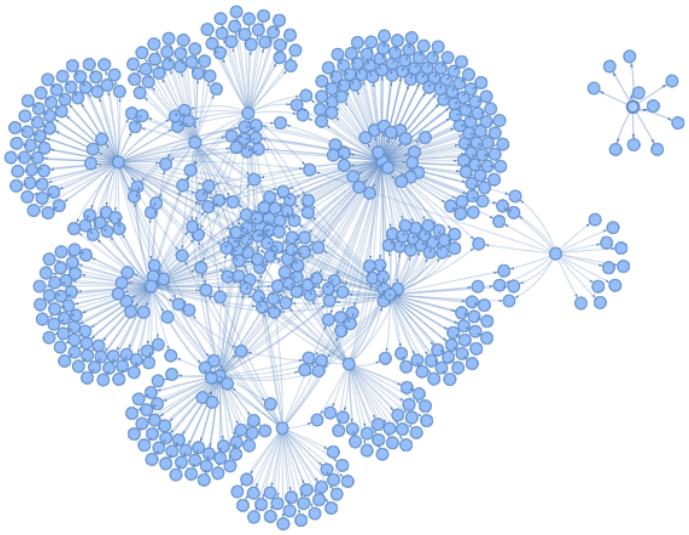
- ACTED_IN, DIRECTED, RATED, IN_GENRE, and RELEASED

Properties**:**

- Nodes and relationships contain attributes such as imdbRating, year, budget, languages, and rating.

Here is a sample of the knowledge graph that has the RATED relationship. This shows the user named 'Stacy Grant' and what movies they gave a rating too:

The following image displays the first 800 nodes in the same knowledge graph. The reason for choosing only 800 values is because if we tried to display all nodes the process would crash the visualization:



Relevance to Project: The Neo4j knowledge graph provides a multi-relational structure

connecting movies to genres, actors, directors, and users through various relationships. This structure is inherently supportive of multi-hop reasoning.

Rather than executing explicit multi-hop Cypher queries at retrieval time, we collected all relevant single-hop facts first. These were transformed into semantic triplets and used to build a FAISS index. During inference, the LLM performs implicit multi-hop reasoning by synthesizing retrieved facts to answer user queries.

IV. PROBLEM DEFINITION

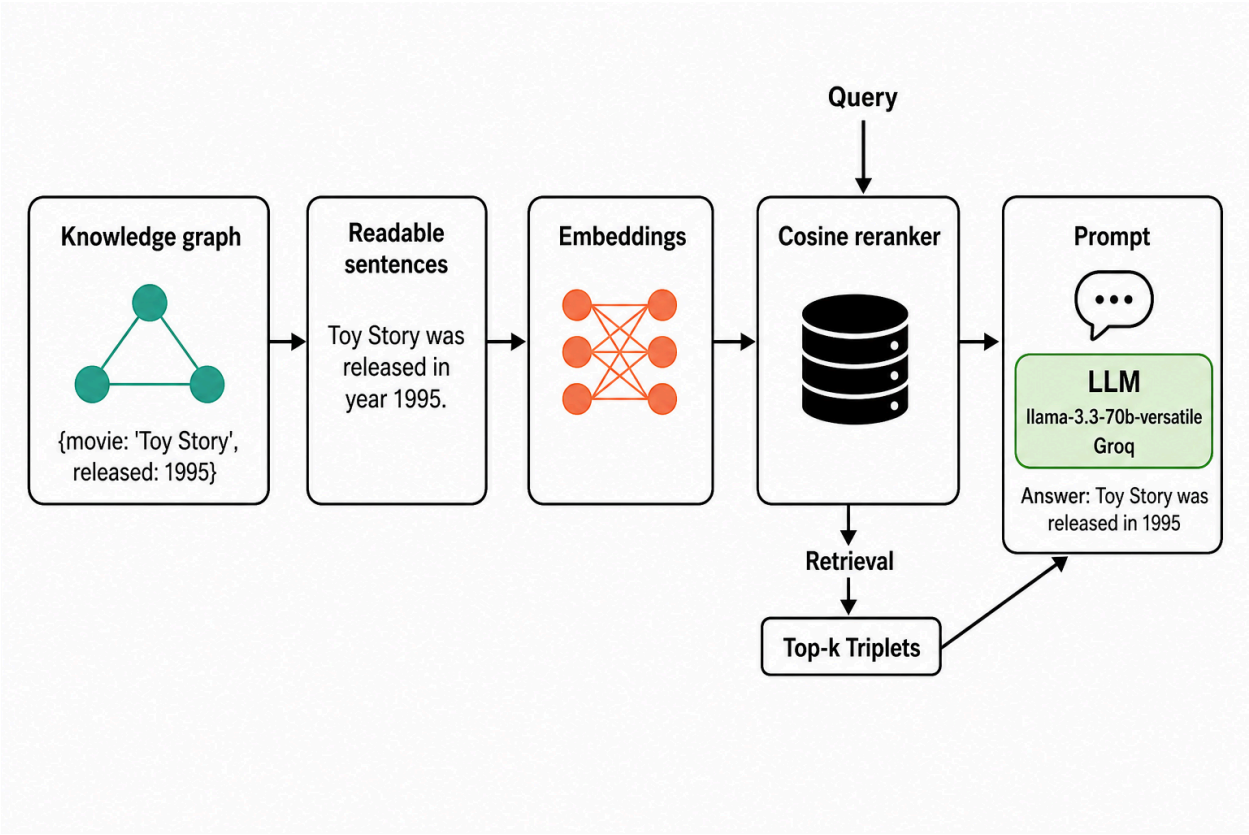
The project addresses two core challenges:

- Bias Mitigation: Traditional models tend to favor popular movies. Our graph-based system recommends based on entity relationships, reducing reliance on popularity.
- Cold-Start Resolution: For new users or movies, knowledge graph connections enable relationship inference and candidate recommendation generation.

Our system predicts potential user-movie interactions using graph structure and semantic triplets and justifies recommendations through natural language, enhancing both utility and explainability.

V. METHODOLOGY OVERVIEW

Our architecture consists of five main stages, as illustrated below:



1. Creating Readable Triplets: Extract subject-predicate-object triplets, ("Toy Story", "was released in year", "1995"), from Neo4j's graph.
2. Embedding Generation: Convert natural language triplets into dense vector embeddings using the all-MiniLM-L6-v2 sentence transformer.
3. FAISS Index Construction: Store all 384-dimensional sentence embeddings into a FAISS index to support efficient similarity search.
4. Retrieval with Re-ranking: For each user query, retrieve top-k triplets using L2 distance in FAISS, and re-rank using cosine similarity.
5. RAG Chain Creation: Use the LangChain RetrievalQA module with Groq's LLaMA3-70B model to generate answers grounded in retrieved triplets.

VI. IMPLEMENTATION

A. Knowledge Graph Construction:

```
from langchain_community.graphs import Neo4jGraph
kg = Neo4jGraph(url="bolt://demo.neo4jlabs.com:7687",
username="recommendations", password="recommendations",
database="recommendations")
```

Cypher queries were used to extract movie properties, actor/director relationships, genres, and user ratings.

B. Creating Readable Triplets:

```
triplets.append(("Toy Story", "was released in year", "1995"))
sentence = f"{subj} {rel} {obj}"
```

A total of 229,894 triplets were created and converted into natural language sentences.

C. Embedding Generation:

```
from sentence_transformers import SentenceTransformer
encoder_model = SentenceTransformer('all-MiniLM-L6-v2')
triplet_embeddings = encoder_model.encode(triplet_sentences,
batch_size=64, show_progress_bar=True)
```

Generated 384-dimensional embeddings for each sentence.

D. FAISS Index Construction:

```
import faiss
index = faiss.IndexFlatL2(384)
index.add(np.array(triplet_embeddings).astype('float32'))
```

Indexed all embeddings to enable fast vector similarity search.

E. Retrieval with Re-ranking

```
from sentence_transformers import util
similarities = util.cos_sim(query_embedding, triplet_embeddings)[0]
```

A hybrid retriever used FAISS with L2 distance for initial retrieval and cosine similarity for

semantic re-ranking.

F. RAG Chain Creation:

```
from langchain_groq import ChatGroq
from langchain.chains import RetrievalQA
qa_chain = RetrievalQA.from_chain_type(
    llm=ChatGroq(model_name="llama3-70b-8192", temperature=0),
    retriever=vectorstore.as_retriever(),
    chain_type="stuff",
    chain_type_kwargs={"prompt": prompt_template}
)
```

Constructed a RetrievalQA chain using Groq's LLaMA3 model to generate fact-grounded recommendations.

These steps help create a recommendation engine that is accurate, scalable, and interpretable

These implementation choices ensure efficient retrieval, scalability, and improved semantic understanding of user queries.

VII. EXPERIMENTS AND RESULTS

We evaluated our system with queries related to genre, actor, and release year to test the reasoning across multiple attributes.

A. Sample Queries and Answers:

Query: "Find movies acted by Leonardo DiCaprio released after 2000."

- Our Model's Answer: Gangs of New York, Catch Me If You Can, The Aviator, Blood Diamond, The Departed, etc.
- Cypher Ground Truth: Matched 10 out of 14 correct entries.

Query: "Can you recommend me a recent movie with Adventure genre and rating greater than 3 with actor Leonardo DiCaprio."

- Our Model's Answer: The Revenant (2015)
- Cypher Ground Truth: The Revenant, Blood Diamond, The Beach
- Match: Yes
- B. Evaluation Metrics and Performance Analysis
-

- To quantitatively evaluate the effectiveness of our RAG-based recommendation system, we adopt standard information retrieval (IR) metrics: Precision@K, Recall@K, and F1-score@K.
- Precision@K measures the proportion of relevant movies among the top-K retrieved results.
- Recall@K measures the proportion of relevant movies successfully retrieved out of all relevant items in the ground truth.
- F1-score@K is the harmonic mean of precision and recall, providing a balanced evaluation of the system's performance.
- We evaluated the system across multiple query types, including genre-based, actor-based, and constraint-based queries. Ground truth results were obtained using Cypher queries executed directly on the Neo4j knowledge graph.
- Performance Results:
- Query 1: List adventure movies released after 2000
- Precision@10: 0.60
- Recall@10: 0.01
- F1-score@10: 0.02

- Query 2: Find movies acted by Leonardo DiCaprio released after 2000
- Precision@10: 0.70
- Recall@10: 0.50
- F1-score@10: 0.58

- Overall System Performance:
- Average Precision@10: 0.65
- Average Recall@10: 0.26
- Average F1-score@10: 0.30

- Analysis:
- The system demonstrates strong precision, indicating that the retrieved recommendations are highly relevant. However, recall remains relatively low, suggesting that the system does not retrieve all possible relevant items.
- This behavior can be attributed to several factors:
- The use of top-K retrieval limits the number of candidate results
- Re-ranking prioritizes highly confident matches, reducing diversity
- Sparse user interaction data in the knowledge graph affects coverage

- As a result, the system prioritizes accuracy over completeness, making it well-suited for scenarios where high-quality recommendations are more important than exhaustive retrieval.
- Extended Evaluation:
- To further validate the system, we conducted experiments on a larger set of queries across multiple language models. Results indicate a trade-off between performance and efficiency: smaller models provide faster responses with moderate accuracy, while larger models improve answer quality at the cost of increased latency.
- These findings highlight important considerations for real-world deployment, where both response time and recommendation quality must be balanced.

VIII. TASK DISTRIBUTION AND CONTRIBUTIONS

Bharath Kumar A
Designed the RAG pipeline, FAISS retrieval workflow, embedding process, evaluation metrics, and contributed to report writing and analysis.

Venkata Siva Sai Krishna Prasad Yedupati::

Worked on Neo4j knowledge graph connection, Cypher-based data extraction, triplet generation, preprocessing, and system testing.

Sriya Verma Saripella:

Worked on user interface/demo preparation, experiment validation, presentation slides, screenshots, and documentation.

All team members contributed to system integration, testing, final presentation, and report

review.

VIII. CONCLUSION AND FUTURE WORK

Our KG and RAG approach enhances the explainability and personalization of movie recommendations. By embedding graph-based relationships and using RAG for natural language generation, we achieved strong precision with interpretable outputs.

Key Takeaways:

- Knowledge Graphs mitigate popularity bias and enable recommendations for unseen entities.
- FAISS and re-ranking support efficient and semantically accurate retrieval.
- RAG enables natural language justification for recommendations.

Future Work:

- Improve recall through hybrid retrievers such as combining plot text and KG triplets.
- Fine-tune re-ranking with supervised learning.
- Extend to larger graphs such as IMDb and TMDB.

IX. REFERENCES

[1] P. Lewis *et al.*, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," *arXiv.org*, Apr. 12, 2021. https://arxiv.org/abs/2005.11401

[2] A. Hogan *et al.*, "Knowledge Graphs," *ACM Computing Surveys*, vol. 54, no. 4, pp. 1–37, May 2022, doi: https://doi.org/10.1145/3447772.

[3] "KG-Retriever: Efficient Knowledge Indexing for Retrieval-Augmented Large Language Models," *Arxiv.org*, 2023. https://arxiv.org/html/2412.05547v1 (accessed Mar. 13, 2025).

[4] L. S. Nair and J. Cheriyan, “Multi-Featured Movie Recommendation Using Knowledge Graph,” Jan. 2023, doi: https://doi.org/10.1109/idciot56793.2023.10053435.

[5] Z. Xu *et al.*, “Retrieval-Augmented Generation with Knowledge Graphs for Customer Service Question An-swering,” 2024, doi: https://doi.org/10.1145/3626772.3661370.

[6] “Example datasets - Getting Started,” *Neo4j Graph Data Platform*. https://neo4j.com/docs/getting-started/appendix/example-data/